

GUIA MAESTRA DEL STACK TECNICO

Tienda online + Bot de WhatsApp de pedidos

GitHub · Supabase · Railway · Meta WhatsApp Cloud API · Resend

Documentado a partir de la implementacion real de **The Cap Store Online** (tienda de gorras colombianas).
Pensada como plantilla replicable para el proximo proyecto que arranque desde cero.

Elaborado: septiembre de 2026

Todas las claves, tokens y contraseñas de este documento son marcadores de ejemplo (PLACEHOLDER) —
ninguna credencial real del proyecto esta incluida.

Indice

0. Como usar esta guia	4
1. Punto de partida: un correo y un WhatsApp, nada mas	4
2. GitHub — control de versiones y sitio web	5
2.1 Crear la cuenta y el repositorio	5
2.2 Publicar el sitio con GitHub Pages (gratis)	5
2.3 Si el bot es un backend Node aparte: SEGUNDO repositorio	5
3. Supabase — base de datos	5
3.1 Crear el proyecto	5
3.2 Diseñar la tabla pensando en mas de un canal desde el inicio	6
3.3 Esquema real usado (registro_chat)	6
3.4 Seguridad: RLS y las dos claves de API	7
3.5 Verificar el esquema real antes de asumir	7
4. Railway — hosting del backend	7
4.1 Desplegar	7
4.2 El puerto tiene que coincidir (gotcha real, costo: horas de debugging)	8
4.3 Variables de entorno	8
4.4 Desactivar "Enable Serverless" mientras se prueba WhatsApp	8
4.5 Anotar la URL publica	8
5. Meta WhatsApp Cloud API	8
5.1 Por que Meta directo y no Twilio	8
5.2 Cuenta propia del negocio vs. cuenta de un cliente	8
5.3 Crear la app en developers.facebook.com	9
5.4 Numero de prueba, token temporal y webhook	9
5.5 Gotcha: mensajes entregados (doble check) pero el webhook nunca dispara	10
5.6 Token permanente (System User) — obligatorio para produccion	10
5.7 Migrar al numero real del negocio (irreversible)	11
6. Resend — correo de notificaciones	11
6.1 Por que no Gmail/SMTP directo (costo real: una tarde de debugging)	11
6.2 Crear la cuenta y la API key	11
6.3 Uso en el codigo (fire-and-forget)	12

7. Arquitectura del bot: maquina de estados	12
7.1 Reglas de disciplina que evitaron bugs reales	13
7.2 Persistencia de pedidos	13
7.3 Limitacion conocida, sin resolver: sesiones solo en memoria	13
8. Pruebas y verificacion de punta a punta	13
8.1 Orden recomendado (evita perder tiempo)	13
8.2 Como se verifico realmente en este proyecto	13
8.3 Senales de fallo y donde mirar	14
9. Costos y limites gratuitos (resumen)	14
10. Bitacora real de errores y soluciones	14
11. Checklist maestro para el proximo proyecto	16

0. Como usar esta guía

Esta guía documenta, paso a paso y con evidencia real (no generica), como se construyo el stack completo de The Cap Store Online: sitio web + bot de pedidos por WhatsApp. Cada seccion cubre una plataforma: que es, como se crea, como se conecta con las demas, que fallo durante la implementacion real y como se resolvió, y que parte es gratuita.

NOTA — El objetivo no es "una API en abstracto", sino el camino ya recorrido: los errores documentados aqui (tokens vencidos, puertos que no coinciden, RLS mal configurado, SMTP bloqueado) ya ocurrieron una vez en este proyecto real. Evitarlos la segunda vez es el valor de este documento.

Estructura del documento:

- **Seccion 1** — el punto de partida minimo: un correo y un WhatsApp nuevos.
- **Secciones 2 a 6** — cada plataforma en el orden real en que conviene crearlas.
- **Seccion 7** — como queda armado el bot por dentro (arquitectura de software).
- **Seccion 8** — como se prueba todo de punta a punta antes de dar por listo.
- **Seccion 9** — tabla resumen de que es gratis y donde estan los limites.
- **Seccion 10** — bitacora real de errores encontrados y su solucion exacta.
- **Seccion 11** — checklist maestro para copiar y usar en el proximo proyecto.

1. Punto de partida: un correo y un WhatsApp, nada mas

Cada proyecto nuevo (tienda, bot, negocio de un conocido) arranca de exactamente estas dos cosas, antes de tocar cualquier plataforma:

- **Un correo electronico propio del negocio** (ej. Gmail nuevo, tipo minegocio@gmail.com). No usar el correo personal del dueño del negocio: todo lo demás (GitHub, Supabase, Railway, Meta Business, Resend) se registra con este correo, para que el negocio sea dueño de sus propios activos desde el día uno.
- **Un numero de WhatsApp**. Puede ser el numero personal/de negocio que ya usa el cliente. No hace falta comprar un numero nuevo: Meta ofrece un numero de PRUEBA gratuito para desarrollar y probar todo el flujo; el numero real del negocio solo se conecta al final, cuando el bot ya funciona de punta a punta (ver seccion 5.7).

RIESGO — Migrar el numero real a la Cloud API de Meta desconecta ese numero de la app movil de WhatsApp Business de forma **irreversible** (se pierde el historial de chats de esa app). Confirmar esto explicitamente con el dueño del negocio antes de hacerlo — nunca de forma automatica ni "para ahorrar un paso".

Con correo y WhatsApp listos, el orden recomendado (validado en este proyecto, ver seccion 9 para el porque) es: **GitHub** → **Supabase** → **Railway** → **Meta WhatsApp** → **Resend**. Construir la base de datos y el hosting antes de tocar Meta evita configurar un webhook que apunte a un servidor que todavia no existe.

2. GitHub — control de versiones y sitio web

2.1 Crear la cuenta y el repositorio

- Crear la cuenta en github.com con el correo del proyecto (sección 1).
- Crear un repositorio nuevo (público, si se va a usar GitHub Pages gratuito) con el nombre del proyecto.
- Clonar el repositorio localmente con `git clone` desde el primer commit, en vez de subir archivos manualmente desde la interfaz web.

ADVERTENCIA — En este proyecto, el repositorio del sitio se inició subiendo archivos uno por uno con el botón "Add files via upload" de GitHub (visible todavía en los primeros commits del historial), sin repositorio git local. Eso funciona para publicar rápido, pero complica llevar un historial de cambios ordenado y dificulta usar Claude Code u otras herramientas de terminal sobre el proyecto. Para el próximo proyecto: **clonar con git desde el día 1**, aunque sea un solo archivo `index.html`.

2.2 Publicar el sitio con GitHub Pages (gratis)

- Settings → Pages → Source: la rama `main`, carpeta raíz.
- Queda publicado en `https://<usuario>.github.io/<repo>/`.
- Esta URL sirve, entre otras cosas, como "sitio web" y como URL de política de privacidad que Meta exige para publicar la app de WhatsApp (sección 5.4).

2.3 Si el bot es un backend Node aparte: SEGUNDO repositorio

NOTA — Si el bot de WhatsApp corre en Railway como un servidor Node separado, crear un **segundo repositorio** dedicado solo a ese backend (en este proyecto: `chatbot-server-CapStore`). Railway despliega exactamente el contenido de ese repo, y **nada fuera de él** — ver el gotcha de las imágenes en la sección 10.

- Ese segundo repo necesita su propio `package.json`, su propio `.gitignore` (excluir `.env` y `node_modules`), y TODO archivo que el código lea en tiempo de ejecución (imágenes, catálogos, plantillas) tiene que vivir dentro de esa misma carpeta/repositorio, nunca en una carpeta hermana del proyecto padre.

3. Supabase — base de datos

3.1 Crear el proyecto

- supabase.com → New Project, con el correo del proyecto.
- Elegir la región más cercana al mercado del negocio (ej. la región de AWS más cercana a Colombia) y una contraseña de base de datos fuerte (se guarda aparte, no es la misma que las API keys).

GRATIS / OK — El nivel gratuito de Supabase (usado en este proyecto) alcanza para un proyecto chico/mediano: base de datos Postgres, Auth, Storage y API REST/Realtime incluidos. Ver límites exactos vigentes en supabase.com/pricing antes de asumir cifras, cambian con el tiempo.

3.2 Diseñar la tabla pensando en mas de un canal desde el inicio

Lección real de este proyecto: la tabla de pedidos se diseñó pensando solo en el canal web, y hubo que agregar después una columna canal con una migración SQL correctiva (ver `db/migration_001_canal_unificado.sql`). Para el próximo proyecto, incluir desde el `CREATE TABLE` inicial:

```
canal VARCHAR(10) NOT NULL DEFAULT 'web'
CHECK (canal IN ('web', 'whatsapp'))
```

También decidir de una vez si la tabla es **"una fila por producto"** (lo usado aquí: `numero_pedido` se repite en varias filas, sin `UNIQUE`) o **"una fila por pedido completo"** con el detalle de productos en una columna JSON o una tabla hija. Cambiar de un modelo al otro a mitad de proyecto obliga a migraciones SQL correctivas como la de este caso.

3.3 Esquema real usado (registro_chat)

```
CREATE TABLE IF NOT EXISTS registro_chat (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fecha TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Producto
  referencia VARCHAR(20),
  nombre_producto VARCHAR(200),
  coleccion VARCHAR(20) CHECK (coleccion IN ('agropecuario', 'colombia', 'luxury')),
  precio INTEGER,
  -- Contacto del cliente
  nombre VARCHAR(100),
  celular VARCHAR(15),
  direccion TEXT,
  ciudad VARCHAR(100),
  departamento VARCHAR(100),
  correo VARCHAR(150),
  -- Resultado
  canal VARCHAR(10) NOT NULL DEFAULT 'web' CHECK (canal IN ('web', 'whatsapp')),
  estado VARCHAR(12) NOT NULL
  CHECK (estado IN ('pendiente', 'confirmado', 'cancelado')),
  numero_pedido VARCHAR(20), -- sin UNIQUE: varias filas comparten uno
  pago VARCHAR(60) DEFAULT 'Contra entrega en efectivo'
);
```

Regla de negocio de este modelo: cada fila es un evento de compra o intento con datos de contacto. No se guarda nada si el usuario solo explora sin dar información. `estado = confirmado` implica `numero_pedido` presente; `cancelado` implica que ya se habían capturado datos antes de abandonar.

3.4 Seguridad: RLS y las dos claves de API

Supabase usa dos tipos de llave (esquema vigente desde 2025):

Clave	Uso	Donde va
publishable (sb_publishable_...)	Equivalente a la vieja "anon". Publica.	Frontend / navegador (script.js del sitio)
secret (sb_secret_...)	Equivalente a la vieja "service_role". Bypassa RLS.	Solo Variables de entorno del backend (F

```
ALTER TABLE registro_chat ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY "anon_insert_registro"  
ON registro_chat FOR INSERT TO anon WITH CHECK (true);  
-- La clave publica del sitio SOLO puede insertar, nunca leer ni modificar.
```

RIESGO — Fallo real detectado y corregido el 2026-09-12: se creo una VISTA (pedidos_confirmados) para consultar pedidos desde el panel de Supabase. Una vista corre por defecto con los permisos de su DUENO (el rol postgres), **ignorando el RLS** de la tabla de abajo. Resultado: la clave publica anon (la misma del sitio web) podía leer nombre, celular y direccion de TODOS los pedidos confirmados a traves de esa vista, sin que el RLS de registro_chat lo impidiera. Se confirmo el problema con una prueba real (insert + lectura con la clave publica) antes de corregirlo.

```
ALTER VIEW pedidos_confirmados SET (security_invoker = true);  
REVOKE ALL ON pedidos_confirmados FROM anon, authenticated;
```

ADVERTENCIA — Leccion permanente: toda vista sobre una tabla con RLS necesita `security_invoker = true` explicito, o hereda los permisos del dueño y expone los datos igual que si el RLS no existiera. Verificar esto en CUALQUIER vista nueva que se cree sobre una tabla protegida.

3.5 Verificar el esquema real antes de asumir

El .sql del repositorio puede no coincidir con la tabla real si alguna columna se agrego a mano desde el panel de Supabase. Antes de correr una migracion nueva, verificar siempre:

```
SELECT column_name, data_type FROM information_schema.columns  
WHERE table_name = 'registro_chat';
```

4. Railway — hosting del backend

4.1 Desplegar

- railway.app → New Project → Deploy from GitHub repo → elegir el repositorio del backend (el segundo repo de la seccion 2.3, si aplica).
- Railway detecta package.json y corre `npm install && npm start` automaticamente.

4.2 El puerto tiene que coincidir (gotcha real, costo: horas de debugging)

ADVERTENCIA — Railway asigna un **Target Port** fijo al dominio publico (Settings → Networking), normalmente 8080. Si la variable de entorno que define el puerto de escucha de la app (API_PORT, PORT, etc.) tiene un valor distinto, las peticiones se cuelgan hasta el timeout: se ve en Network Logs como status **499** con duracion de **~30 segundos**, y NO aparece nada en los logs de la aplicacion, porque el request nunca llega a llamar al código. Solucion: poner la variable de entorno con el mismo valor que el "Target Port".

4.3 Variables de entorno

Railway **no lee el .env del repositorio**. Cada variable se carga aparte en la pestaña **Variables** del servicio (usar "Raw Editor" para pegar todas de una vez). Tras guardar queda un cambio pendiente — hay que darle clic explicito a **"Deploy"** para que tome efecto.

4.4 Desactivar "Enable Serverless" mientras se prueba WhatsApp

ADVERTENCIA — En Settings → Deploy, la opcion "Enable Serverless" escala el contenedor a cero tras inactividad para ahorrar recursos/costo. Railway encola las peticiones mientras el contenedor duerme, pero **Meta no espera ese "cold start"** y da el webhook por fallido. Mientras se depura el flujo de WhatsApp, mantener el servicio siempre activo; evaluar reactivar "Serverless" mas adelante segun el trafico real y el plan de Railway.

4.5 Anotar la URL publica

La URL `https://<servicio>.up.railway.app` es la `PUBLIC_BASE_URL` que se necesita para el webhook de Meta y para las URLs de imagenes que el bot envia.

5. Meta WhatsApp Cloud API

5.1 Por que Meta directo y no Twilio

GRATIS / OK — Se evaluo Twilio primero y se descarto: las cuentas **Trial** nuevas exigen usar plantillas de contenido (ContentSid) incluso para respuestas de texto libre dentro de la ventana de 24h (error real: **21654 "ContentSid Required"**), y bloquean Settings/Templates/Logs detras de un muro de "agregar saldo". Meta Cloud API directa es gratis para desarrollo y no tiene esa restriccion.

5.2 Cuenta propia del negocio vs. cuenta de un cliente

Si el negocio es propio, se crea todo con la cuenta personal de Facebook, sin friccion. **Si el proyecto es para un tercero**, el patron cambia:

- No hace falta una Pagina de Facebook clasica: Meta exige un **Business Portfolio** (antes "Business Manager"), gratis y sin verificacion de documentos para desarrollar.
- Los activos (Business Portfolio, WABA, numero, app) deben quedar a nombre del **cliente**, no del desarrollador — si no, el día que cambie de desarrollador todo queda atado a una cuenta ajena.
- Caso mas comun (negocio chico sin nada creado): el cliente crea su propio Business Portfolio con su cuenta, y agrega al desarrollador como colaborador en Configuracion del negocio → Usuarios → Personas con rol Administrador. La app y la WABA se crean estando "parado" en el portfolio del cliente, para que queden de su propiedad desde el origen.
- Caso con dos portfolios ya existentes (agencia + cliente establecido): se usa Socios → Solicitar acceso a los activos de un socio pidiendo el Business Manager ID del cliente.

5.3 Crear la app en developers.facebook.com

- "Mis apps" → "Crear app". Nombre de la app: **no puede contener "WhatsApp"** ni otras marcas de Meta (Face, Book, Insta, Gram) — se rechaza con error. Usar algo como <Negocio> Bot Pedidos.
- Caso de uso: "Conectarte con los clientes a traves de WhatsApp".
- Crear/usar un Business Portfolio; elegir "Verificar mas tarde" cuando lo pida (la verificacion de documentos solo hace falta para publicar datos de terceros, no para desarrollo).

5.4 Numero de prueba, token temporal y webhook

Dentro de la app: Casos de uso → Personalizar → "Conectar en WhatsApp" → "Integrar con la API" (no "Convertirte en socio", eso es para agencias/BSP).

- "Paso 1. Pruebalo" da un numero de prueba gratis, un Phone Number ID, un WhatsApp Business Account ID y un token de acceso **siempre temporal (~23-24h)**, sin importar cuantas veces se regenere desde ahí.
- Ese numero de prueba solo envia a **hasta 5 destinatarios verificados** (numero + codigo OTP) — suficiente para probar el flujo completo del bot.

Configurar el webhook (dentro de "Conectar en WhatsApp" → panel izquierdo → "Otras herramientas" → "Herramientas", o en "Paso 2: Configuracion de produccion → Configurar webhooks" si el boton anterior no lleva ahí):

```
URL de devolucion de llamada: {PUBLIC_BASE_URL}/whatsapp/webhook
Token de verificacion: una cadena inventada por ti (no la da Meta)
-> debe coincidir con META_VERIFY_TOKEN del backend
```

```
// Verificación GET que exige Meta al guardar el webhook
api.get('/whatsapp/webhook', (req, res) => {
  const { 'hub.mode': mode, 'hub.verify_token': token, 'hub.challenge': challenge } =
    req.query;
  if (mode === 'subscribe' && token === process.env.META_VERIFY_TOKEN) {
    return res.status(200).send(challenge);
  }
  res.sendStatus(403);
});

// Mensajes entrantes
api.post('/whatsapp/webhook', express.json(), async (req, res) => {
  res.sendStatus(200); // responder rapido, procesar aparte
  const message = req.body?.entry?.[0]?.changes?.[0]?.value?.messages?.[0];
  if (!message) return; // puede ser un evento de "status", no de mensaje
});
```

5.5 Gotcha: mensajes entregados (doble check) pero el webhook nunca dispara

RIESGO — Si el destinatario ve el mensaje entregado en WhatsApp, pero Railway (Network Logs) no recibe ningún POST, y ya se descarto lo obvio (URL/token, campo messages suscrito, servidor despierto), revisar en este orden: **(1) la app sin publicar** — Meta no entrega webhooks reales a apps no publicadas, aunque el número sea de prueba; publicar exige icono 1024x1024, categoría y URL de política de privacidad (puede ser una página simple en GitHub Pages, ver sección 2.2). **(2) La app no está suscrita al WABA a nivel de cuenta** — distinto de suscribir el campo messages a nivel de app.

```
# Confirmar que la app aparece suscrita a la cuenta de WhatsApp (WABA):
GET https://graph.facebook.com/v20.0/{WABA_ID}/subscribed_apps?access_token={TOKEN}

# Si sale vacío o falta la app, suscribirla:
POST https://graph.facebook.com/v20.0/{WABA_ID}/subscribed_apps?access_token={TOKEN}
```

5.6 Token permanente (System User) — obligatorio para producción

ADVERTENCIA — El token de "Paso 1. Pruébalo" vence cada ~24h. Si el bot deja de responder de un día para otro sin cambios de código, revisar primero los Deploy Logs buscando Meta API 401 ... code 190 ... OAuthException (token vencido) antes de asumir otra causa.

- business.facebook.com/settings → Usuarios → Usuarios del sistema → Agregar. El nombre del perfil es estricto: rechaza demasiados guiones y mayúsculas seguidas (ej. capstore-bot-system-user falla; Capstorebotssystem funciona). Rol: Administrador.
- En la página del usuario del sistema → "Agregar activos": tipo **Apps** (permiso "Administrar app") y tipo **Cuentas de WhatsApp** (permiso "Todo"). El usuario del sistema **no hereda acceso a WABAs nuevas** automáticamente.

- Boton "Generar nuevo token" **desde esta pagina** (no desde "Paso 1. Prueballo", que siempre da temporal aunque se vea igual). Scopes: whatsapp_business_management + whatsapp_business_messaging. Expiracion: "Nunca".

```
# Verificar antes de confiar en el token:  
GET https://graph.facebook.com/debug_token?input_token={TOKEN}&access_token={TOKEN}  
# Debe devolver: "type":"SYSTEM_USER", "expires_at":0
```

5.7 Migrar al numero real del negocio (irreversible)

RIESGO — Confirmar explícitamente con el dueño del negocio antes de este paso: **el numero debe desconectarse por completo de la app movil de WhatsApp Business para poder registrarse en la Cloud API**, con perdida irreversible del historial de chats de esa app. Validar primero TODO el flujo con el numero de prueba.

- "Paso 2: Configuración de producción" → "Agregar numero nuevo" → llenar info de negocio y perfil de WhatsApp Business.
- Si Meta dice que el numero "ya esta registrado con una cuenta de WhatsApp": la unica salida (integración directa, sin BSP) es eliminar la cuenta desde la app movil (Configuración → Cuenta → "Eliminar mi cuenta"), esperar ~3 minutos y reintentar.
- Esto crea una **WABA nueva**, distinta de la del numero de prueba. Repetir en ella: suscripción de la app (5.5) y acceso del usuario del sistema (5.6) — nada se hereda automáticamente.
- Registrar con el código SMS/llamada de 6 dígitos. Activar el toggle "Suscribir webhooks" junto al numero (equivalente en UI al POST manual de subscribed_apps).
- Actualizar META_PHONE_NUMBER_ID en Railway → Deploy. Probar con un mensaje real antes de dar la migración por cerrada.

6. Resend — correo de notificaciones

6.1 Por que no Gmail/SMTP directo (costo real: una tarde de debugging)

RIESGO — Primer intento: nodemailer contra Gmail por SMTP (puertos 465 y 587). Railway **bloquea las conexiones salientes por SMTP** — ambos puertos daban "Connection timeout" en producción (aunque funcionaban en local). Se probaron ambos puertos y variantes de trust proxy antes de identificar que el bloqueo era de la plataforma de hosting, no del código. Solución: reemplazar SMTP por la **API HTTP de Resend** (resend.com), que no depende de un puerto SMTP saliente.

6.2 Crear la cuenta y la API key

- resend.com → crear cuenta con el correo del proyecto.
- API Keys → crear una nueva → guardarla como RESEND_API_KEY en las variables de entorno del backend (Railway), nunca en el código ni en el repo.

GRATIS / OK — Nivel gratuito de Resend: se puede enviar usando el dominio de pruebas `onboarding@resend.dev` sin verificar un dominio propio — suficiente para notificaciones internas del negocio (ej. avisar un pedido nuevo a la casilla del propio negocio). Para enviar como `pedidos@tunegocio.com` hace falta verificar el dominio propio (registros DNS) — revisar límites vigentes en resend.com/pricing.

6.3 Uso en el código (fire-and-forget)

```
const { Resend } = require('resend');
const resend = new Resend(process.env.RESEND_API_KEY);

async function sendOrderEmail(order) {
  // Se llama SIN await desde el flujo del pedido, para no bloquear
  // la respuesta al cliente mientras se envía el correo.
  const { error } = await resend.emails.send({
    from: 'Mi Negocio <onboarding@resend.dev>',
    to: 'notificaciones@minegocio.com',
    subject: `Nuevo pedido de ${order.cliente.nombre}`,
    html: '...',
  });
  if (error) throw new Error(error.message);
}
```

NOTA — El correo de notificación siempre llega al mismo buzón del negocio, sin importar el canal (web o WhatsApp) por el que entro el pedido — es la forma en que el dueño del negocio se entera de un pedido nuevo sin tener que revisar Supabase manualmente.

7. Arquitectura del bot: maquina de estados

Cada conversación activa (identificada por el número de WhatsApp del cliente) vive como un objeto en un Map en memoria del proceso Node, con esta forma general:

```
sessions[chatId] = {
  state: 'menu' | 'collection' | 'viewing_product' | 'cart_view' |
  'datos_nombre' | 'datos_telefono' | ... | 'confirmacion' | 'mayoreo_*',
  collection: null | '<categoria>',
  currentProduct: null | { id, name, image, price },
  cart: [ { id, name, price, collection } ],
  datos: { nombre, telefono, direccion, ciudad, depto, correo },
  referencia: null | 'PREFIJO-XXXXXX', // generado al mostrar el resumen
  prevState: null | string, // para volver tras un desvío
}
```

7.1 Reglas de disciplina que evitaron bugs reales

- Escribir "menu"/"hola"/"inicio" vuelve al menu **sin perder el carrito** (funcion separada `volverAlMenu()`). Vaciar el carrito por completo (`resetSession()`) queda reservado para pedido confirmado, cancelado, o la palabra explicita "vaciar".
- Dentro de un estado de captura de datos (direccion, telefono, etc.), esas palabras de reinicio exigen coincidencia EXACTA, no `startsWith` — evita que una direccion como "Hipodromo, Bogota..." dispare un reinicio accidental por empezar con "hi".
- Todas las comparaciones de texto se normalizan (minusculas, sin tildes) antes de comparar contra listas fijas de respuestas afirmativas/negativas aceptadas.

7.2 Persistencia de pedidos

Cada pedido confirmado inserta **una fila por producto del carrito** en Supabase, todas compartiendo el mismo `numero_pedido` (coherente con el modelo "un evento de compra por fila" de la seccion 3.3). Se valida si un pedido es duplicado consultando Supabase por telefono + los ultimos 10 minutos, en vez de mantener un archivo JSON local.

7.3 Limitacion conocida, sin resolver: sesiones solo en memoria

ADVERTENCIA — Un pedido a medio armar (carrito con productos, todavia sin confirmar) se pierde por completo si el proceso de Railway se reinicia — por ejemplo, al desplegar un cambio de codigo — porque nada queda persistido hasta que el cliente llega al paso de "confirmar". Si esto se vuelve un problema recurrente en produccion, la solucion de fondo es persistir el estado de la sesion en Supabase en vez de en un Map del proceso.

8. Pruebas y verificacion de punta a punta

8.1 Orden recomendado (evita perder tiempo)

- [] Supabase: credenciales listas + esquema real verificado con `information_schema`.
- [] Backend: variables de entorno completas en Railway, push a GitHub, deploy activo.
- [] Railway: puerto de la variable de entorno = Target Port (Settings -> Networking).
- [] Meta: app creada, webhook apuntando al dominio de Railway YA activo (no antes).
- [] Probar con el numero de PRUEBA (hasta 5 destinatarios) antes de tocar el numero real.

8.2 Como se verifico realmente en este proyecto

Se hizo un pedido real de principio a fin por WhatsApp y se confirmo directamente en Supabase (tabla `registro_chat`, filtrando por `canal='whatsapp'`) que el pedido llego completo y bien etiquetado, en vez de asumir que "si el bot respondio, ya quedo guardado".

GRATIS / OK — El fallo de seguridad de la sección 3.4 (vista exponiendo datos a la clave anon) se descubrió con exactamente este método: una prueba real de insert + lectura usando la clave pública del sitio, no una revisión visual del SQL. Para cualquier tabla con RLS, la única verificación confiable es intentar leer/escribir con la clave que usaría un atacante real.

8.3 Señales de fallo y donde mirar

Sintoma	Causa probable	Donde revisar
Status 499, ~30s de espera	Puerto de la variable != Target Port	Railway -> Network Logs
Mensaje entregado (check azul) pero webhook no dispara	App no publica WABA no suscrita	Meta -> Configurar webhooks / subscribed_apps
Bot deja de responder de un día para otro	Token temporal vencido (~24h)	Railway -> Deploy Logs, buscar "code 190"
404/401 al pedir una imagen pública	Asset fuera del repo que Railway despliega	curl -I a la URL pública de la imagen
Correo de notificación no llega	RESEND_API_KEY ausente o SMTP bloqueado	Deploy Logs, buscar el log de éxito/error del envío

9. Costos y límites gratuitos (resumen)

Plataforma	Que se uso	Costo en este proyecto
GitHub	Repo público + GitHub Pages	Gratis
Supabase	Proyecto free tier (Postgres + API)	Gratis (verificar límites vigentes en supabase.com/pricing)
Railway	Un servicio Node desplegado	Plan de pago por uso; revisar el plan activo en Railway -> Billing
Meta WhatsApp Cloud API	Número de prueba + número real, integración	Gratis para desarrollo; conversaciones de negocio con costo
Resend	Dominio de pruebas onboarding@resend.dev	Gratis dentro del límite de envíos del plan free (ver resend.com/pricing)

ADVERTENCIA — Los precios y límites de estas plataformas cambian con frecuencia. Esta tabla documenta que TIER se usó en este proyecto, no cifras exactas de dólares/límites — confirmar siempre el límite vigente en el sitio oficial antes de presupuestar un proyecto nuevo.

10. Bitacora real de errores y soluciones

Registro cronológico de los problemas que realmente ocurrieron durante la implementación, para no repetirlos en el próximo proyecto.

Twilio Trial exige plantillas hasta para texto libre

Que paso: Error 21654 "ContentSid Required" al responder texto libre dentro de la ventana de 24h; Settings/Templates/Logs bloqueados tras muro de "upgrade".

Solucion: Se abandono Twilio y se migro a Meta Cloud API directa (seccion 5).

Fotos de producto nunca llegaban por WhatsApp

Que paso: El codigo buscaba las imagenes en una carpeta hermana (. ./images) fuera del repo que Railway realmente despliega. Nunca funciono desde que existia el bot.

Solucion: Las imagenes se movieron dentro del propio repo del bot (chatbot/images/), sincronizadas desde la carpeta del sitio con un script.

El carrito se vaciaba sin avisar

Que paso: Escribir "menu"/"hola"/"0" para volver al menu llamaba a `resetSession()`, que borra tambien el carrito.

Solucion: Se separo "volver al menu" (preserva carrito) de "vaciar todo" (accion explicita). Ver seccion 7.1.

Webhook de Meta no disparaba pese a mensajes entregados

Que paso: La suscripcion al campo "messages" es a nivel de APP, pero falta suscribir la app al WABA a nivel de CUENTA (paso no documentado en la UI normal de Meta).

Solucion: POST manual a `/waba_id/subscribed_apps`, confirmado con un GET al mismo endpoint. Ver seccion 5.5.

Bot dejo de responder tras ~24h sin cambios de codigo

Que paso: El token de acceso en uso era el temporal de "Paso 1. Pruebalo" (vence cada ~24h). Log real: Meta API 401, code 190, OAuthException.

Solucion: Se creo un usuario del sistema (System User) con token permanente (`expires_at: 0`). Ver seccion 5.6.

Puerto de Railway no coincidia

Que paso: La variable de entorno del puerto de la app tenia un valor distinto al Target Port configurado en Railway -> Networking.

Solucion: Se iguala la variable de entorno al Target Port real. Sintoma: status 499 sin nada en los logs de la app. Ver seccion 4.2.

SMTP (Gmail) fallaba con "Connection timeout" en produccion

Que paso: Railway bloquea conexiones salientes por SMTP (puertos 465 y 587), aunque funcionaba perfecto en local.

Solucion: Se reemplazo nodemailer/SMTP por la API HTTP de Resend. Ver seccion 6.1.

Vista de Supabase exponía datos de pedidos a la clave publica

Que paso: La vista pedidos_confirmados corria con permisos de su dueño (postgres), ignorando el RLS de la tabla base.

Solucion: Se aplico `security_invoker = true` y `REVOKE ALL` a "anon"/"authenticated". Confirmado con prueba real. Ver seccion 3.4.

Numero real ya "registrado" al migrar de la app movil

Que paso: Meta rechaza registrar un numero que sigue activo en la app movil de WhatsApp Business; no hay opcion de "coexistencia" sin un BSP.

Solucion: Se elimino la cuenta desde la app movil (irreversible, con consentimiento previo) y se reintento tras ~3 minutos. Ver seccion 5.7.

11. Checklist maestro para el proximo proyecto

Antes de crear nada

- [] Correo propio del negocio creado (no el personal del dueño).
- [] Definido si el proyecto es cuenta propia o de un cliente/tercero (seccion 5.2).

GitHub

- [] Cuenta y repositorio(s) creados con el correo del proyecto.
- [] Clonado con git desde el dia 1 (no "Add files via upload").
- [] GitHub Pages activado si el sitio es estatico.
- [] Si el bot es un backend separado: repo propio, con TODOS los assets que necesita en tiempo de ejecucion dentro de ese mismo repo.

Supabase

- [] Tabla disenada con columna "canal" desde el CREATE TABLE inicial.
- [] Decidido el modelo de filas (una por producto vs. una por pedido) desde el inicio.
- [] RLS activado; policy de solo INSERT para "anon".
- [] Toda vista sobre tabla con RLS tiene `security_invoker = true + REVOKE ALL`.
- [] Clave secret SOLO en variables de entorno del backend, nunca en el repo.

Railway

- [] Variable de puerto de la app = Target Port (Settings -> Networking).
- [] Variables de entorno cargadas en la pestana Variables (no en .env del repo).
- [] "Enable Serverless" desactivado mientras se prueba el flujo de WhatsApp.
- [] URL publica anotada como PUBLIC_BASE_URL.

Meta WhatsApp Cloud API

- [] App creada (nombre sin "WhatsApp" ni otras marcas de Meta).
- [] Webhook configurado y verificado, apuntando a Railway ya activo.
- [] App suscrita al WABA a nivel de CUENTA (POST subscribed_apps), no solo el campo "messages" a nivel de app.
- [] App publicada (icono, categoria, URL de politica de privacidad).
- [] Token permanente de usuario del sistema generado y cargado en Railway.
- [] Flujo probado end-to-end con el numero de prueba (hasta 5 destinatarios).
- [] Numero real migrado SOLO con confirmacion explicita del dueno del negocio, y probado con un mensaje real antes de cerrar la migracion.

Resend (correo)

- [] Cuenta creada con el correo del proyecto, API key en variables de entorno.
- [] Confirmado que el envio funciona en Railway (no solo en local) antes de asumirlo.

Verificacion final

- [] Un pedido real de punta a punta, confirmado directamente en Supabase (no solo "el bot respondio bien").
- [] Prueba de seguridad: intentar leer datos sensibles con la clave publica del sitio y confirmar que RLS/vistas lo bloquean.

Fuentes de este documento: CLAUDE.md del proyecto, la guia interna "whatsapp-cloud-supabase-railway", el checklist "GUIA_IMPLEMENTACION_WHATSAPP_SUPABASE_RAILWAY.md", el codigo real (mailer.js, schema.sql, migration_001) y el historial de git de ambos repositorios (sitio y bot).